

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – ERROR ANALYSIS

|                  |                                                                                                                            |               |                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------|
| Course Code:     | DSA03                                                                                                                      | Course Title: | Natural Language Processing        |
| CO Assessed:     | CO2 – Manipulation                                                                                                         | Assessment:   | Assessment Tool 3 – ERROR ANALYSIS |
| Weightage:       | 30% of CIA                                                                                                                 |               |                                    |
| CO5 Statement:   | Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3) |               |                                    |
| Student Name:    | T Jyothi Reddy                                                                                                             | Reg. No.:     | 192472079                          |
| Section / Batch: | 2024                                                                                                                       | Date:         |                                    |

### Code of Conduct

I T Jyothi Reddy (192472079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T Jyothi Reddy

### Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

| Section / Topic                                     | Focus                                                                                                                                                                                                                                                | Q Nos |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Inflectional Morphology and Derivational Morphology | Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.               | Q1    |
| Inflectional Morphology and Derivational Morphology | Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.                                                                                    | Q2    |
| Finite-State Morphological Parsing                  | Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.                                      | Q3    |
| Finite-State Morphological Parsing                  | Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.                                            | Q4    |
| Porter Stemmer and Applications of Stemming in NLP  | Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.             | Q5    |
| Porter Stemmer and Applications of Stemming in NLP  | Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems. | Q6    |
| Porter Stemmer and Applications of Stemming in NLP  | Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.          | Q7    |

## Annexure A – Error Analysis

---

### Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

#### Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

### Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

#### Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

### Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

#### Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
  - Original text
  - Tokens
  - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

### Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```

def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))

```

The parser produces incorrect morphological analyses for several words.

#### Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
  - Regular plurals
  - Words ending in -es
  - Words ending in -ies
  - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

#### Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]

```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

#### Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
  - Vocabulary size before correction
  - Vocabulary size after correction
  - Classification accuracy before correction
  - Classification accuracy after correction
  - Processing time

6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

#### Rubrics for Calculation

| Criteria                              | Weightage  | Level 4 – Exemplary                                                 | Level 3 – Proficient                                 | Level 2 – Developing                                  | Level 1 – Beginning                           |
|---------------------------------------|------------|---------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|
| <b>Error Identification</b>           | <b>20%</b> | Accurately identifies all errors with clear understanding of issues | Identifies most errors with minor omissions          | Identifies limited errors; some are missed            | Fails to identify key errors                  |
| <b>Root Cause Analysis</b>            | <b>30%</b> | Clearly explains underlying causes with strong technical reasoning  | Explains causes with moderate clarity                | Partial explanation; weak reasoning                   | Unable to identify causes of errors           |
| <b>Correction &amp; Justification</b> | <b>25%</b> | Provides correct solutions with strong justification and logic      | Provides correct corrections with some justification | Corrections are partially correct or weakly justified | Incorrect or no corrections provided          |
| <b>Application of Concepts</b>        | <b>15%</b> | Effectively applies relevant concepts to analyze and resolve errors | Applies concepts with minor errors                   | Limited application; requires guidance                | Unable to apply concepts                      |
| <b>Clarity &amp; Presentation</b>     | <b>10%</b> | Presents analysis clearly, logically, and systematically            | Generally clear with minor issues                    | Lacks clarity or proper structure                     | Poor presentation and difficult to understand |

| Q. No. / Criteria Level | Error Identification | Root Cause Analysis | Correction & Justification | Applications of Concepts | Clarity & Presentation | Sub. Total | Total % |
|-------------------------|----------------------|---------------------|----------------------------|--------------------------|------------------------|------------|---------|
| 1                       | 4                    | 4                   | 4                          | 3                        | 3                      | 18         | 90      |
| 2                       | 3                    | 3                   | 4                          | 4                        | 4                      | 18         |         |
| 3                       | 4                    | 3                   | 3                          | 4                        | 4                      | 18         |         |
| 4                       | 3                    | 4                   | 3                          | 4                        | 4                      | 18         |         |
| 5                       | 4                    | 4                   | 4                          | 3                        | 3                      | 18         |         |

| Faculty In-charge     | Course Coordinator | Program Director |
|-----------------------|--------------------|------------------|
| Dr.T. Kumaragurubaran |                    |                  |
| Signature: _____      | Signature: _____   | Signature: _____ |
| Date: _____           | Date: _____        | Date: _____      |

## 1. Morphological Analysis

The correct morphological analyses are:

| Word        | Prefix | Root  | Suffix | Affix Type                             |
|-------------|--------|-------|--------|----------------------------------------|
| treatment   | —      | treat | -ment  | Derivational                           |
| treatable   | —      | treat | -able  | Derivational                           |
| retreatment | re-    | treat | -ment  | re-: Derivational, -ment: Derivational |
| treated     | —      | treat | -ed    | Inflectional                           |
| untreated   | un-    | treat | -ed    | un-: Derivational, -ed: Inflectional   |

The incorrect analysis occurs when the system removes affixes without considering their morphological function. For example, **retreatment** should not simply be reduced to *retreat* because it contains two meaningful morphological layers:

**re- + treat + -ment**

A corrected strategy should identify prefixes and suffixes separately, determine their order, classify them as derivational or inflectional, and then identify the root.

This improves biomedical information retrieval because related forms such as **treatment** and **retreatment** can be connected through the common root **treat**, while meaningful distinctions such as *re-* and *un-* are preserved.

### Python Program

```
words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]

prefixes = {
    "re": "Derivational",
    "un": "Derivational"
}

suffixes = {
    "ment": "Derivational",
    "able": "Derivational",
    "ed": "Inflectional"
}

def analyze(word):
    prefix = ""
    suffix = ""
    root = word

    for p in prefixes:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break

    for s in suffixes:
        if root.endswith(s):
            suffix = s
            root = root[:-len(s)]
            break

    return prefix, root, suffix
```

```

for word in words:
    prefix, root, suffix = analyze(word)

    print("\nWord:", word)
    print("Prefix:", prefix if prefix else "None")
    print("Root:", root)
    print("Suffix:", suffix if suffix else "None")

    if prefix:
        print("Prefix Type:", prefixes[prefix])

    if suffix:
        print("Suffix Type:", suffixes[suffix])

```

## 2. Finite-State Morphological Parsing

The correct morphological structures are:

| Word         | Correct Analysis     |
|--------------|----------------------|
| replayed     | re- + play + -ed     |
| unhappier    | un- + happy + -er    |
| disconnected | dis- + connect + -ed |
| players      | play + -er + -s      |
| restarting   | re- + start + -ing   |
| unreadable   | un- + read + -able   |

A suitable finite-state structure is:

```

START
↓
PREFIX
↓
ROOT
↓
SUFFIX
↓
SUFFIX
↓
END

```

The transitions should allow optional prefixes and multiple suffixes. This allows forms such as:

```

re + play + ed
play + er + s
un + read + able
re + start + ing

```

For a word of length  $n$ , a basic deterministic finite-state parser operates approximately in  $O(n)$  time.

### Python Program

```

words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]

```

```

def parser(word):

    prefix = ""
    root = word
    suffixes = []

    for p in ["re", "un", "dis"]:
        if root.startswith(p):
            prefix = p
            root = root[len(p):]
            break

    if root.endswith("ing"):
        suffixes.insert(0, "ing")
        root = root[:-3]

    elif root.endswith("ed"):
        suffixes.insert(0, "ed")
        root = root[:-2]

    if root.endswith("er"):
        suffixes.insert(0, "er")
        root = root[:-2]

    if root.endswith("able"):
        suffixes.insert(0, "able")
        root = root[:-4]

    if root.endswith("s"):
        suffixes.insert(0, "s")
        root = root[:-1]

    if root == "happi":
        root = "happy"

    return prefix, root, suffixes

for word in words:
    prefix, root, suffixes = parser(word)

    print("\nWord:", word)
    print("Prefix:", prefix if prefix else "None")
    print("Root:", root)
    print("Suffixes:", suffixes)

```

#### **Expected Output**

```

replayed
Prefix: re
Root: play
Suffixes: ['ed']

```

```

unhappier
Prefix: un
Root: happy
Suffixes: ['er']

```

disconnected  
Prefix: dis  
Root: connect  
Suffixes: ['ed']

players  
Prefix: None  
Root: play  
Suffixes: ['er', 's']

restarting  
Prefix: re  
Root: start  
Suffixes: ['ing']

unreadable  
Prefix: un  
Root: read  
Suffixes: ['able']  
Parsing accuracy is:

**Accuracy = Correct Analyses / Total Analyses × 100**

For example, 6 correct analyses out of 6 gives:

**Accuracy = 100%**

### 3. Stemming Program

The corrected program tokenizes the complete text first and then applies stemming to each individual word.

```
import pandas as pd
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
```

```
# nltk.download('punkt')
```

```
ps = PorterStemmer()
```

```
data = pd.read_csv("news.csv")
```

```
def process_text(text):
    tokens = word_tokenize(str(text))

    stems = []

    for word in tokens:
        if word.isalpha():
            stems.append(ps.stem(word.lower()))

    return stems
```

```
data["Tokens"] = data["Text"].apply(
    lambda x: word_tokenize(str(x))
)
```

```
data["Stemmed Tokens"] = data["Text"].apply(
    process_text
)
```

```
print(data[["Text", "Tokens", "Stemmed Tokens"]].head())
```

### Example

#### Original:

The players are playing football.

#### Tokens:

```
['The', 'players', 'are', 'playing', 'football']
```

#### Stemmed Tokens:

```
['the', 'player', 'are', 'play', 'footbal']
```

#### Twenty Stemming Cases

| Word          | Stem    | Type                        |
|---------------|---------|-----------------------------|
| connected     | connect | Inflectional                |
| connecting    | connect | Inflectional                |
| connections   | connect | Derivational + Inflectional |
| connectivity  | connect | Derivational                |
| studies       | studi   | Inflectional                |
| studied       | studi   | Inflectional                |
| studying      | studi   | Inflectional                |
| study         | studi   | Derivational                |
| organization  | organ   | Derivational                |
| organizations | organ   | Derivational + Inflectional |
| organized     | organ   | Derivational + Inflectional |
| organizer     | organ   | Derivational                |
| relational    | relat   | Derivational                |
| relation      | relat   | Derivational                |
| happiness     | happi   | Derivational                |
| happily       | happili | Derivational                |
| hopeful       | hope    | Derivational                |
| kindness      | kind    | Derivational                |
| running       | run     | Inflectional                |
| easily        | easili  | Derivational                |

The corrected preprocessing produces tokens and stems separately, making the data suitable for further NLP processing.

## 4. Plural-Noun Morphological Parser

```
def parser(word):
```

```
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "feet": "foot",
        "teeth": "tooth"
    }
```

```
    if word in irregular:
        return irregular[word], "Plural Noun"
```

```
    if word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun"
```

```

if word.endswith("es"):
    return word[:-2], "Plural Noun"

if word.endswith("s"):
    return word[:-1], "Plural Noun"

return word, "Singular"

```

```

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

```

```

for w in words:
    print(w, "->", parser(w))

```

### Output

```

cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
books -> ('book', 'Plural Noun')

```

The parser handles:

Regular: books → book

-es: boxes → box

-ies: cities → city

Irregular: children → child

A rule-based parser is efficient, but it requires additional rules for irregular and exceptional forms.

## 5. Morphological Preprocessing for Feature Extraction

The correct processing order is:

Documents

↓

Tokenization

↓

Normalization / Stemming

↓

Feature Extraction

↓

Machine Learning

### Corrected Program

```

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

```

```

stemmer = PorterStemmer()

```

```

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

```

```

def stem_document(document):
    words = document.lower().split()

    return " ".join(
        stemmer.stem(word)
        for word in words
    )

processed_documents = [
    stem_document(doc)
    for doc in documents
]

print("Original Documents:")

for doc in documents:
    print(doc)

print("\nProcessed Documents:")

for doc in processed_documents:
    print(doc)

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(processed_documents)

print("\nVocabulary:")
print(vectorizer.get_feature_names_out())

print("\nFeature Matrix:")
print(X.toarray())

```

### Example Normalization

| Original     | Normalized |
|--------------|------------|
| connected    | connect    |
| connection   | connect    |
| connecting   | connect    |
| connectivity | connect    |
| studies      | studi      |
| studied      | studi      |
| studying     | studi      |
| organize     | organ      |
| organized    | organ      |
| organizer    | organ      |

### 20 Newsgroups Experiment

```

import time
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

```

```

data = fetch_20newsgroups(
    subset="train",
    categories=[
        "sci.space",
        "rec.sport.baseball"
    ],
    remove=("headers", "footers", "quotes")
)

documents = data.data
labels = data.target

def stem_document(text):
    words = text.lower().split()

    return " ".join(
        stemmer.stem(word)
        for word in words
    )

# Before normalization
start = time.time()

vectorizer1 = CountVectorizer(max_features=5000)
X1 = vectorizer1.fit_transform(documents)

time_before = time.time() - start
vocab_before = len(vectorizer1.get_feature_names_out())

# After normalization
start = time.time()

processed = [
    stem_document(doc)
    for doc in documents
]

vectorizer2 = CountVectorizer(max_features=5000)
X2 = vectorizer2.fit_transform(processed)

time_after = time.time() - start
vocab_after = len(vectorizer2.get_feature_names_out())

print("Vocabulary before:", vocab_before)
print("Vocabulary after:", vocab_after)

print("Processing time before:",
      round(time_before, 4), "seconds")

print("Processing time after:",
      round(time_after, 4), "seconds")

```

## Classification Accuracy

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
train_text, test_text, train_y, test_y = train_test_split(
    documents,
    labels,
    test_size=0.2,
    random_state=42
)
```

```
# Before normalization
```

```
vectorizer1 = CountVectorizer(max_features=5000)
```

```
X_train1 = vectorizer1.fit_transform(train_text)
```

```
X_test1 = vectorizer1.transform(test_text)
```

```
model1 = LogisticRegression(max_iter=1000)
```

```
model1.fit(X_train1, train_y)
```

```
pred1 = model1.predict(X_test1)
```

```
accuracy_before = accuracy_score(test_y, pred1)
```

```
# After normalization
```

```
train_stemmed = [
    stem_document(x) for x in train_text
]
```

```
test_stemmed = [
    stem_document(x) for x in test_text
]
```

```
vectorizer2 = CountVectorizer(max_features=5000)
```

```
X_train2 = vectorizer2.fit_transform(train_stemmed)
```

```
X_test2 = vectorizer2.transform(test_stemmed)
```

```
model2 = LogisticRegression(max_iter=1000)
```

```
model2.fit(X_train2, train_y)
```

```
pred2 = model2.predict(X_test2)
```

```
accuracy_after = accuracy_score(test_y, pred2)
```

```
print("Accuracy before:",
      round(accuracy_before * 100, 2), "%")
```

```
print("Accuracy after:",
      round(accuracy_after * 100, 2), "%")
```